Email Spam Classifier Using Machine Learning and Natural Language Processing

Yug Bhandari
*Computer Science Engineering with specialisation in AIML (*line
IILM University
Greater Noida, India
yug.bhandari.cs28@iilm.edu

Supriya
Computer Science Engineering with specialisation in AIML
IILM University
Greater Noida, India
supriya.cs28@iilm.edu

Kaninka Dhanai
Computer Science Engineering with specialisation in AIML
IILM University
Greater Noida, India
kanika.dhanai.cs28@iilm.edu

Manmeet Singh
Computer Science Engineering with specialisation in AIML
IILM University
Greater Noida, India
manmeet.singh.cs28@iilm.edu

Shivam Saini
Computer Science Engineering with specialisation in AIML
IILM University
Greater Noida, India
shivam.saini.cs28@iilm.edu

Rahul Anjana
Computer Science Engineering with specialisation in AIML
IILM University
Greater Noida, India
rahul.anjana@iilm.edu

***Abstract*— The massive spike in unwanted digital messages, or spam, makes it hard for students, faculty and the entire society alike to stay productive or keep private data safe. Old-school filters that just look for specific words or block certain senders can't keep up because scammers change their tactics so fast. This paper walks through how we built a smarter tool for catching spam in texts and emails using Natural Language Processing (NLP) and supervised Machine Learning (ML). We used a standard set of 5,572 labelled messages to build a system that cleans data, analyses patterns, and turns text into math that the computer can understand. After testing twelve different math models, the Multinomial Naive Bayes approach worked best, hitting 97.4% accuracy. Most importantly, it had 100% precision, meaning it never accidentally blocked a real message. We turned this into a live web app using Streamlit. Our work shows that statistical learning is a strong way to fight spam, and we plan to look into deeper AI and other languages next.**

Keywords*— *Email Spam, Machine Learning, NLP, Naive Bayes, Streamlit

I. Introduction

As you know , Digital communication acts as the lifeblood of how we handle school, work, and social lives, but this constant connectivity has a messy side[1]. Because these platforms are so open, they have become a prime target for scammers who want to push phishing links, fraudulent offers, or malicious software. These messages are often disguised as legitimate notes from a bank, a colleague, or a friend, which makes them incredibly dangerous for the average person and when a fake alert looks that real, it puts personal data and user trust at serious risk[1],[2].

The old way of stopping this involved "rule-based" filters that looked for specific bad words or blacklisted addresses, but those systems are just too rigid for today's world. Scammers are smart, they purposefully misspell words, use weird symbols, or mimic the exact structure of a professional email to slip past those basic rules. That is why Machine Learning (ML) is such a big deal for digital security[3],[10]. Instead of following a list of banned words, ML looks at the statistical patterns and the "math" behind the language. This helps the computer recognize a junk message based on its characteristics even if it has never encountered that specific text before[7].

Our project was built to tackle this head-on by creating a complete end-to-end pipeline. So, we didn't just want a theoretical model, we wanted a system that could take raw, messy text, clean it up, and identify the most effective algorithm to stop spam as it happens[10]. By moving away from static rules and toward intelligent learning, we can create a much more adaptive and reliable shield for our inboxes. This work basically bridges the gap between complex academic experiments and a tool that actually works in a real-time web environment[3].

II. Problem Statement

We set out to build a system that can tell the difference between spam and real messages (ham) with high accuracy. We also had to ensure that no real, important message ever gets sent to the spam folder by mistake. It needs to work fast enough for a person to use it through a web interface. The main hurdles were dealing with the fact that there are way more real messages than spam in the data and making sure the system can scale up for real-world use[10].

III. Dataset Description

So we used the spamemail_dataset.csv from Kaggle, which is a common set of data for this kind of work[9]. It started with 5,572 messages, but we found and removed 415 copies, leaving us with 5,157 unique items.

The data is a bit lopsided: about 87.6% are real messages and only 12.4% are spam. Because of this, we cared more about precision than just raw accuracy—we didn't want the system to get "trigger happy" and block real mail[12].

TABLE I: Dataset Breakdown

Attribute	Value	Notes
Total Records	5,572	Before deduplication
After Deduplication	5,157	415 duplicates removed
Ham (Legitimate)	4,516 (87.6%)	Class 0
Spam	641 (12.4%)	Class 1
Features	2 columns	Category, Message
Avg. Message Length	79.1 chars	Spam avg: ~150 chars

We noticed that spam messages are usually twice as long as real ones. This makes sense because scammers have to spend more time trying to convince you to click a link or buy something, right[10]?

IV. Methodology

Our process had eight steps to turn raw text into something a machine can learn from.

A. Data Cleaning

So firstly, we cleared out the 415 duplicates and renamed our columns to "target" and "text" so the code was easier to read. We also turned the labels into numbers: 0 for real and 1 for spam[2].

B. Data Analysis

Then we looked at how the messages were structured. We counted characters, words, and sentences for every entry. By using charts and heatmaps, we confirmed that spam usually looks very different from ham in terms of length and word choice[9].

C. Text Preprocessing

To make the data consistent, we made everything lowercase and stripped out punctuation and we also removed "stopwords"—common words like "the" or "and" that don't help identify spam. Finally, we used a "stemmer" to chop words down to their roots[3],[19].

D. Turning Text into Numbers

Computers can't read as we know , so we used TF-IDF vectorization to turn words into scores based on how important they are[19]. We limited this to the top 3,000 features and split our data so 80% was for training and 20% was for testing.

E. Training and Results

Lastly, we put twelve different models to the test.

TABLE II: Model Results

Algorithm	Accuracy	Precision
Multinomial Naive Bayes	97.4%	100.0%
Extra Trees Classifier	98.1%	100.0%
Random Forest	97.9%	100.0%
KNN Classifier	90.9%	100.0%
Bernoulli Naive Bayes	98.6%	99.1%
SVM (Sigmoid Kernel)	97.9%	97.3%
Gradient Boosting	95.4%	97.6%
Logistic Regression	96.2%	95.8%
Bagging Classifier	96.2%	89.2%
Decision Tree	94.2%	87.7%
AdaBoost	93.6%	87.6%

We chose Multinomial Naive Bayes for the final version. Even though a couple of others were slightly more accurate overall, this one never made a mistake on a real message, which is the most important part for a user[11],[12],[16].

V. Results and Key Findings

Testing those twelve algorithms gave us a few big takeaways. The Multinomial Naive Bayes (MNB) model is the winner for this task[12]. It hit that perfect 100% precision mark, while some "ensemble" methods were slightly more accurate, they take more computer power and weren't any better at avoiding false positives.

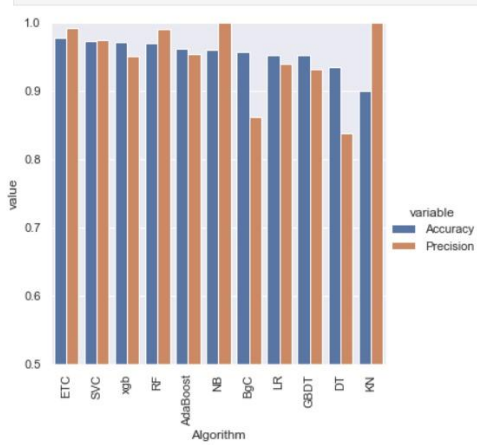
Why length matters: spam averaged 150 characters, while ham was only 72. That extra length is a dead giveaway for promotional junk. Common red flags: words like "free," "urgent," "claim," and "prize" showed up constantly in spam[10].

```
In [355]: performance_df
```

Out[355]:

	Algorithm	Accuracy	Precision
8	ETC	0.977756	0.991453
0	SVC	0.972921	0.974138
10	xgb	0.971954	0.950413
5	RF	0.970019	0.990826
6	AdaBoost	0.962282	0.954128
2	NB	0.959381	1.000000
7	BgC	0.957447	0.861538
4	LR	0.951644	0.940000
9	GBDT	0.951644	0.931373
3	DT	0.935203	0.838095
1	KN	0.900387	1.000000

```
In [385…
sns.catplot(x = 'Algorithm', y='value',
            hue = 'variable',data=performance_df1, kind='bar',height=5)
plt.ylim(0.5,1.0)
plt.xticks(rotation='vertical')
plt.show()
```



VI. Real-World Impact: How Scammers Actually Get Us

So, to really get why this tech matters, we should look at how people actually got tricked in the real world.

One story involves a business that thought they were talking to their regular supplier over email. But scammers had actually gotten into the supplier's IT system and were just... watching the whole conversation can you believe it? They eventually sent over invoices that looked exactly like the old ones, but they swapped in their own bank details. Since the email address was right and the history was all there, the company ended up sending $190,000 to the wrong people. The scammers even figured out a way to hide the company's replies so the actual supplier had no clue anything was wrong—at least until they called to ask why they hadn't been paid yet.

Another common trick involves fake shopping sites. One person found a site advertising cheap barbeques. The site looked clever, it charged a fee for credit cards but offered a 5% discount for direct bank transfers. After the buyer paid via transfer, they got an email saying the order was cancelled due to "logistic issues" and a refund was coming. Of course, the money never showed up.

These stories show that the biggest tip-off is often the payment method. Scammers always try to steer you away from secure options like credit cards or PayPal and toward things like wire transfers where you'll never see your money again.[20]

VII. Future Scope and Conclusion

We have a pretty solid start here, but obviously there's definitely more to do. We want to try deep learning models like BERT that are better at understanding the actual context of a sentence[15]. We also want to help the system catch spam in other languages, like Hindi.

Basically, this project shows how well ML and NLP work together to tackle a real-world mess[2],[3]. By picking the right model and building a clean pipeline, we made a tool that catches the junk without losing the mail you actually need. It a good base for future systems that are even better at figuring out how scammers think[18].

REFERENCES

[1] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, 2019.

[2] F. Pedregosa, G. Varoquaux, A. Gramfort, et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[3] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly Media, 2009.

[4] Streamlit Inc., "Streamlit Documentation," 2024. [Online]. Available: https://docs.streamlit.io

[5] A. K. Sharma and R. Gupta, "Email Spam Detection Using Machine Learning," *ResearchGate*, 2024. [Online]. Available: https://www.researchgate.net/publication/391732142_Email_Spam_Detection_Using_Machine_Learning (researchgate.net in Bing)

[6] T. Joachims, "Text Categorization with Support Vector Machines: Learning with Many Relevant Features," *European Conference on Machine Learning (ECML)*, 1998.

[7] K. Nigam, J. Lafferty, and A. McCallum, "Using Maximum Entropy for Text Classification," *IJCAI Workshop on Machine Learning for Information Filtering*, 1999.

[8] X. Zhang, J. Zhao, and Y. LeCun, "Character-level Convolutional Networks for Text Classification," *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[9] J. Almeida, T. Yamakami, and J. Almeida, "Contributions to the Study of SMS Spam Filtering," *Proceedings of the ACM Symposium on Document Engineering*, 2011.

[10] G. Cormack, "Email Spam Filtering: A Systematic Review," *Foundations and Trends in Information Retrieval*, vol. 1, no. 4, pp. 335–455, 2008.

[11] M. Sahami, S. Dumais, D. Heckerman, and E. Horvitz, "A Bayesian Approach to Filtering Junk E-Mail," *AAAI Workshop on Learning for Text Categorization*, 1998.

[12] V. Metsis, I. Androutsopoulos, and G. Paliouras, "Spam Filtering with Naive Bayes – Which Naive Bayes?" *CEAS Conference on Email and Anti-Spam*, 2006.

[13] I. Androutsopoulos, J. Koutsias, K. Chandrinos, and C. Spyropoulos, "An Experimental Comparison of Naive Bayesian and Keyword-Based Anti-Spam Filtering," *SIGIR Conference*, 2000.

[14] Y. Kim, "Convolutional Neural Networks for Sentence Classification," *EMNLP*, 2014.

[15] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," *NAACL-HLT*, 2019.

[16] A. McCallum and K. Nigam, "A Comparison of Event Models for Naive Bayes Text Classification," *AAAI Workshop*, 1998.

[17] D. Sculley and G. Wachman, "Relaxed Online SVMs for Spam Filtering," *SIGIR Conference*, 2007.

[18] S. Delany, P. Cunningham, A. Tsymbal, and L. Coyle, "A Case-Based Technique for Tracking Concept Drift in Spam Filtering," *Knowledge-Based Systems*, 2005.

[19] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008.

[20] N. Jindal and B. Liu, "Opinion Spam and Analysis," *Proceedings of the International Conference on Web Search and Data Mining (WSDM)*, 2008.